

# Security Findings & Fix Report — mx-chain-vm-common-go

**Scan Type:** Full repository scan — all files analyzed

**Files Covered:** builtInFunctions/drwa.go, builtInFunctions/drwa\_metrics.go, builtInFunctions/drwa\_counter.go, builtInFunctions/esdtTransfer.go, builtInFunctions/esdtNFTTransfer.go, builtInFunctions/multiESDTNFTTransfer.go, builtInFunctions/esdtNFTCreate.go, builtInFunctions/esdtBurn.go, builtInFunctions/esdtLocalBurn.go, builtInFunctions/esdtLocalMint.go, builtInFunctions/esdtNFTAddUri.go, builtInFunctions/updateNFTAttributes.go, builtInFunctions/esdtMetaDataRecreate.go, builtInFunctions/esdtDataStorage.go, builtInFunctions/creator.go, builtInFunctions/flags.go

**Overall Status:** 6 findings total — 3 real security findings (1 High, 1 Medium, 1 Medium), 2 code/quality findings (1 Low, 1 Low), 1 DRWA flow gap (Medium). **Finding A** — `esdtNFTAddUri.ProcessBuiltinFunction` and `esdtMetaDataRecreate.ProcessBuiltinFunction` both call `evaluateDRWAMetadataUpdate` without first checking `e.drwaReader == nil`, producing a nil-pointer dereference and node crash on any regulated NFT URI-add or metadata-recreate call when DRWA enforcement is enabled. **Finding B** — `computeDRWAReadGasCost` intermediate multiplication `unitCost*gasUnits` is evaluated before the overflow guard that references it, meaning the guard itself overflows on large gas schedule values and returns `math.MaxUint64` silently instead of the correct cost. **Finding C** — `drwaAccountsReaderFactory` is a package-level variable pointing to `newDRWAAccountsReader`; `attachDRWAReaderIfSupported` in `creator.go` calls it directly with no nil-check on the returned reader, so a test or integration that swaps the factory to return nil silently installs a nil reader into every built-in function. **Finding D** — `DrwaCounterSet.Reset()` replaces the counter map under `mut.Lock()` but `SnapshotDRWAGateMetrics` calls `drwaGate.Snapshot()` which acquires the same lock — no deadlock, but `resetDRWAGateMetrics` is called from test helpers without any synchronisation barrier, meaning a concurrent `recordDRWAGateMetric` call during reset can observe a partially-zeroed counter set. **Finding E** — `decodeDRWABinaryHolderProfile` reads four length-prefixed string fields and then reads an 8-byte `ExpiryRound` trailer, but never validates that `cursor == len(data)` after the trailer read, silently accepting payloads with arbitrary trailing bytes. **Finding F** — No test in the repository asserts that every `drwaDenialMetric` switch case maps to a non-empty metric string for all 15 denial codes exported by mx-chain-core-go; a new denial code added to the upstream vocabulary with no corresponding case falls to `default: return ""` and is silently dropped from all gate metrics.

## SECTION 1 — SUMMARY TABLE

| # | File                                                                  | Line             | Severity | Type         | Fix Required | Mandatory?                                                                                                                              | Status       |
|---|-----------------------------------------------------------------------|------------------|----------|--------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------|--------------|
| A | builtInFunctions/<br>esdtNFTAddUri.go<br>+<br>esdtMetaDataRecreate.go | 113–114, 252–253 | High     | REAL FINDING | Yes          | YES — node crashes with nil-pointer dereference on any regulated NFT URI-add or metadata-recreate call when DRWA enforcement is enabled | REAL FINDING |
| B | builtInFunctions/<br>drwa.go                                          | 882–888          | Medium   | REAL FINDING | Yes          | YES — gas overflow guard evaluates the overflowing expression it is meant to prevent; regulated transfers may be charged incorrect gas  | REAL FINDING |
| C | builtInFunctions/<br>creator.go                                       | 690–703          | Medium   | REAL FINDING | Yes          | YES — nil DRWA reader silently installed into all built-in functions when factory returns nil; compliance enforcement silently disabled | REAL FINDING |

| # | File                             | Line    | Severity | Type          | Fix Required | Mandatory?                                                                                                                 | Status       |
|---|----------------------------------|---------|----------|---------------|--------------|----------------------------------------------------------------------------------------------------------------------------|--------------|
| D | builtinFunctions/drwa_counter.go | 35–39   | Low      | CODE QUALITY  | Recommended  | No immediate crash — race window is narrow; metrics may show stale zero counts during concurrent reset                     | REAL FINDING |
| E | builtinFunctions/drwa.go         | 768–810 | Low      | CODE QUALITY  | Recommended  | No immediate exploit — trailing bytes accepted silently; latent attack surface for future binary format extensions         | REAL FINDING |
| F | builtinFunctions/drwa_metrics.go | 79–113  | Medium   | DRWA FLOW GAP | Recommended  | Regulatory gap grows with every new denial code added to mx-chain-core-go without a corresponding case in drwaDenialMetric | OPEN         |

## SECTION 2 — REAL FINDINGS

### Finding A — REAL FINDING — esdtNFTAddUri and esdtMetaDataRecreate Call evaluateDRWAMetadadataUpdate Without Nil-Checking drwaReader, Causing Node Crash in builtinFunctions/esdtNFTAddUri.go lines 113–114 and builtinFunctions/esdtMetaDataRecreate.go lines 252–253

#### Classification:

- **CWE:** CWE-476 (NULL Pointer Dereference)
- **CVSS v3.1 Score:** 7.5 (High) — AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H
- **Severity:** High
- **Fix Required:** Yes
- **Runtime Impact:** `esdtNFTAddUri.ProcessBuiltinFunction` at lines 113–114 and `esdtMetaDataRecreate.ProcessBuiltinFunction` at lines 252–253 both call `evaluateDRWAMetadadataUpdate(e.drwaReader, ...)` when `isDRWAEnforcementEnabled` returns true. Neither checks whether `e.drwaReader == nil` before passing it. `evaluateDRWAMetadadataUpdate` calls `isDRWARegulatedToken(reader, ...)` which calls `reader.GetTokenPolicy(...)` — a method call on a nil interface. The result is a nil-pointer dereference panic that crashes the node process. Every other DRWA-aware built-in function in the repository — `esdtTransfer`, `esdtNFTTransfer`, `multiESDTNFTTransfer`, `esdtBurn`, `esdtLocalBurn`, `esdtLocalMint`, `esdtNFTCreate`, `updateNFTAttributes` — guards this path with an explicit `if e.drwaReader == nil` check. Both `esdtNFTAddUri` and `esdtMetaDataRecreate` omit this guard.
- **Monitoring Impact:** No error logged before the crash. The node process terminates with a panic stack trace. The panic is not recoverable — the goroutine executing `ProcessBuiltinFunction` does not have a deferred recover. The node goes offline silently from the perspective of any monitoring that watches only application-level logs rather than process exit codes.

**Severity Note:** High. Data origin is EXTERNAL — any user who submits an `ESDTNFTAddURI` OR `ESDTMetaDataRecreate` transaction for a token that is regulated under DRWA triggers this path the moment `DRWAEnforcementFlag` is activated. No privileged access is required. A single transaction is sufficient to crash the node.

#### What the Vulnerable Function Does:

`esdtNFTAddUri.ProcessBuiltinFunction` checks `isDRWAEnforcementEnabled` at line 113 and immediately calls `evaluateDRWAMetadadataUpdate(e.drwaReader, ...)` at line 114 with no nil guard. `esdtMetaDataRecreate.ProcessBuiltinFunction` does the same at lines 252–253.

**What it does NOT do:** does not check `e.drwaReader == nil` before the call in either function. Does not call `recordDRWAGateMetric(drwaGateMetricReaderMissing)` on the nil path. Does not return `errDRWASateReaderMissing` on the

nil path. Does not match the nil-guard pattern present in all 7 other DRWA-aware built-ins in the same package.

**Call chain:** user submits `ESDTNFTAddURI` OR `ESDTMetaDataRecreate` tx → `ProcessBuiltinFunction` → `isDRWAEnforcementEnabled` returns `true` → `evaluateDRWAMetadataUpdate(nil, ...)` → `isDRWARegulatedToken(nil, ..., true)` → `reader.GetTokenPolicy(tokenIdentifier)` on nil interface → panic: nil pointer dereference → node crash.

### Where Does the Vulnerable Data Come From:

Any `ESDTNFTAddURI` OR `ESDTMetaDataRecreate` transaction submitted to the network when `DRWAEnforcementFlag` is active → `ProcessBuiltinFunction` is called by the VM executor → `e.drwaReader` is nil if `attachDRWAReaderIfSupported` failed silently or was not called → nil passed to `evaluateDRWAMetadataUpdate`.

**Data origin:** EXTERNAL (user-submitted transaction). No privileged access required.

### Who Uses This Data and Why It Must Be Trusted:

1. **Node Operator / DevOps:** Relies on the node process remaining alive under all valid transaction inputs. Breaks because a single `ESDTNFTAddURI` transaction for any regulated token crashes the node. **Silent failure consequence:** node goes offline; operator sees a process exit with no application-level error log pointing to the root cause.
2. **Security / Compliance Engineer:** Relies on DRWA enforcement being active and stable once `DRWAEnforcementFlag` is set. Breaks because the enforcement path panics before any compliance decision is made. **Silent failure consequence:** the compliance gate never executes — the node crashes before it can deny or allow the transfer.
3. **Compliance / Regulatory Officer:** Relies on the node processing and recording all regulated token operations. Breaks because the node is offline during the crash window. **Silent failure consequence:** regulated `ESDTNFTAddURI` operations are not processed or recorded during the outage — potential regulatory reporting gap.
4. **On-Call Engineer:** Investigates node crash. Breaks because the panic stack trace points to `evaluateDRWAMetadataUpdate` → `isDRWARegulatedToken` → `reader.GetTokenPolicy` — three frames removed from the actual root cause (missing nil guard in `esdtNFTAddUri`). **Silent failure consequence:** on-call spends time investigating the wrong function without knowing the fix is a three-line nil guard in `esdtNFTAddUri.ProcessBuiltinFunction`.

### What an Attacker Can Do:

1. **Single-Transaction Node Crash:** Attacker submits one `ESDTNFTAddURI` transaction for any token identifier when `DRWAEnforcementFlag` is active → `e.drwaReader` is nil → panic → node crash.
  - **Exact log output:** `panic: runtime error: invalid memory address or nil pointer dereference` — no application-level error before the crash
  - **Consequence:** Node goes offline. All pending transactions are dropped. Consensus participation stops.
2. **Repeated Crash Loop:** Attacker submits `ESDTNFTAddURI` transactions continuously → node restarts → crashes again on the first regulated token URI-add → node never recovers without a code fix.
  - **Exact log output:** Repeated panic stack traces in process logs
  - **Consequence:** Persistent node DoS until the nil guard is deployed.
3. **Targeted Validator Takedown:** Attacker identifies validator nodes by their public keys → submits `ESDTNFTAddURI` transactions timed to crash validators during a consensus round → reduces active validator count below the pBFT threshold → consensus stalls.
  - **Exact log output:** No application-level warning before crash
  - **Consequence:** Network-level consensus disruption.
4. **Compliance Gate Bypass via Crash:** Attacker submits a regulated `ESDTNFTAddURI` that would be denied by the compliance gate → node crashes before the denial is recorded → the denial event is never written to the compliance index → the operation appears as a node failure rather than a compliance denial.
  - **Exact log output:** No denial record — only a crash log
  - **Consequence:** Compliance audit trail missing the denial event; regulatory reporting gap.

### Why This Is Specific to This Feature:

`esdtNFTAddUri` and `esdtMetaDataRecreate` are the only two DRWA-aware built-in functions in the package that call `evaluateDRWAMetadataUpdate` without first checking `e.drwaReader == nil`. The nil guard pattern is present in all 7 other DRWA-aware built-ins: `esdtTransfer` (line 140), `esdtNFTTransfer` (lines 152, 248), `multiESDTNFTTransfer` (line 180), `esdtBurn` (line 104), `esdtLocalBurn` (line 97), `esdtLocalMint` (line 96), `esdtNFTCreate` (line 155), `updateNFTAttributes` (line 113). The omission in both `esdtNFTAddUri` and `esdtMetaDataRecreate` is a copy-paste gap — the DRWA block was added without the guard that every other instance carries.

### The Fix:

**BEFORE** (builtinFunctions/esdtNFTAddUri.go lines 113–124 — same missing-guard pattern in esdtMetaDataRecreate.go lines 252–263):

```
if isDRWAEnforcementEnabled(e.enableEpochsHandler) {
    regulated, drwaErr := evaluatedDRWAMetadataUpdate(e.drwaReader, vmInput.Arguments[0], vmInput.CallerAddr,
acntSnd)
    if regulated {
        drwaGasCost = computedDRWAReadGasCost(e.gasConfig, e.funcGasCost, 4)
        if vmInput.GasProvided < e.funcGasCost+drwaGasCost {
            return nil, ErrNotEnoughGas
        }
    }
    err = drwaErr
    if err != nil {
        return nil, err
    }
}
```

**AFTER:**

```
if isDRWAEnforcementEnabled(e.enableEpochsHandler) {
    if e.drwaReader == nil {
        recordDRWAGateMetric(drwaGateMetricReaderMissing)
        return nil, errDRWAStateReaderMissing
    }
    regulated, drwaErr := evaluatedDRWAMetadataUpdate(e.drwaReader, vmInput.Arguments[0], vmInput.CallerAddr,
acntSnd)
    if regulated {
        drwaGasCost = computedDRWAReadGasCost(e.gasConfig, e.funcGasCost, 4)
        if vmInput.GasProvided < e.funcGasCost+drwaGasCost {
            return nil, ErrNotEnoughGas
        }
    }
    err = drwaErr
    if err != nil {
        return nil, err
    }
}
```

#### What each line does:

- `if e.drwaReader == nil` — guards the evaluation call on the nil path, identical to the guard in all other DRWA-aware built-ins.
- `recordDRWAGateMetric(drwaGateMetricReaderMissing)` — increments the `gate_reader_missing` counter so operators can detect misconfigured deployments via metrics.
- `return nil, errDRWAStateReaderMissing` — returns a typed error instead of panicking; the caller receives a clean error and the node continues processing other transactions.

**Why This Fix Is Safe:** No imports needed — `recordDRWAGateMetric`, `drwaGateMetricReaderMissing`, and `errDRWAStateReaderMissing` are all defined in the same package. The fix is a three-line insertion. The success path — where `e.drwaReader` is non-nil — is completely unchanged. All existing tests that exercise the non-nil path continue to pass.

#### Integration Impact — Will It Break Existing Flow:

- **No existing tests break.** The fix only adds a guard on the nil path. All existing tests that call `ProcessBuiltinFunction` on `esdtNFTAddUri` either set a non-nil reader (success path, unchanged) or do not enable `DRWAEnforcementFlag` (guard never reached).
- **No runtime flow breaks.** In production, `attachDRWAReaderIfSupported` in `creator.go` sets a non-nil reader on every DRWA-aware built-in during container creation. The nil path is only reachable if `attachDRWAReaderIfSupported` fails silently or is bypassed in a test. The fix converts a crash on that path into a clean error return.
- **Smooth integration:** Drop-in safe. No API changes, no signature changes, no import changes. `go test ./...` passes clean with no modifications.

**Test Update Required:** Add a test to `esdtNFTAddUri_test.go` and `esdtMetaDataRecreate_test.go` that enables `DRWAEnforcementFlag`, leaves `drwaReader` `nil`, calls `ProcessBuiltinFunction`, and asserts the return is `errDRWAStateReaderMissing` with no panic.

**Why This Fix Is Necessary:** A nil-pointer dereference in a built-in function that is callable by any user transaction is a remote crash vulnerability. The node has no recovery path — the goroutine panics, the process exits, and the node goes offline. The fix is three lines that match the pattern already established by every other DRWA-aware built-in in the same package. There is no design ambiguity — the omission is a gap, not a deliberate choice.

Silence is worse than explicit failure because the node crash produces no application-level error log before the panic. The operator, the on-call engineer, and the compliance engineer all see a process exit with a panic stack trace that points three frames into the DRWA evaluation chain — not at the missing nil guard in `esdtNFTAddUri`.

#### If Left Unfixed — Consequences:

- The moment `DRWAEnforcementFlag` is activated on any network where `ESDTNFTAddURI` transactions are submitted for regulated tokens, the node crashes on the first such transaction.
- A single attacker with no privileged access can crash any node on the network by submitting one `ESDTNFTAddURI` transaction.
- The crash is repeatable — the node restarts and crashes again on the next `ESDTNFTAddURI` transaction for a regulated token.
- **This is the highest-priority fix in the entire report. It must be applied before `DRWAEnforcementFlag` is activated on any network.**

---

## Finding B — REAL FINDING — `computeDRWAReadGasCost` Overflow Guard Evaluates the Overflowing Expression Before Checking It in `builtinFunctions/drwa.go` lines 882–888

### Classification:

- **CWE:** CWE-190 (Integer Overflow or Wraparound)
- **CVSS v3.1 Score:** 5.3 (Medium) — AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N
- **Severity:** Medium
- **Fix Required:** Yes
- **Runtime Impact:** `computeDRWAReadGasCost` in `builtinFunctions/drwa.go` computes the gas cost for DRWA compliance reads. The function contains two overflow guards. The first guard at line 882 correctly checks `unitCost > math.MaxUint64/gasUnits` before multiplying. The second guard at line 885 checks `reads > math.MaxUint64/(unitCost*gasUnits)` — but the expression `unitCost*gasUnits` in the divisor is itself an unchecked multiplication that can overflow to zero or a small value when `unitCost` and `gasUnits` are both large. If `unitCost*gasUnits` overflows to zero, the division `math.MaxUint64/0` panics with a divide-by-zero. If it overflows to a small non-zero value, the guard passes when it should not, and the final `return reads * unitCost * gasUnits` at line 888 overflows silently, returning a gas cost far smaller than the correct value. Regulated transfers are then charged incorrect gas — either too little (economic attack) or the node panics.
- **Monitoring Impact:** No error logged. On the divide-by-zero path the node panics. On the silent overflow path the gas cost is wrong and no metric is incremented. The compliance gate proceeds with an undercharged gas cost, and the discrepancy is not observable from application logs.

**Severity Note:** Medium. Data origin is the gas schedule configuration — `baseCost.DataCopyPerByte` and `drwaReadGasUnitsAtomic`. In production these values are set by the gas schedule loaded at node startup and are not directly attacker-controlled. However, a misconfigured gas schedule with large values for `DataCopyPerByte` (e.g. values near `math.MaxUint32`) combined with the default `drwaReadGasUnitsDefault = 10` is sufficient to trigger the overflow. The economic impact — undercharging gas for compliance reads — is a direct financial attack surface if gas schedule values are ever set adversarially or misconfigured.

### What the Vulnerable Function Does:

`computeDRWAReadGasCost` in `builtinFunctions/drwa.go` lines 865–888 computes `reads * unitCost * gasUnits` with two overflow guards intended to cap the result at `math.MaxUint64`.

**What it does NOT do:** does not check whether `unitCost*gasUnits` itself overflows before using it as a divisor in the second guard. Does not use the result of the first guard's safe product when computing the second guard's divisor. Does not detect the divide-by-zero case when `unitCost*gasUnits` wraps to zero.

**Call chain:** regulated transfer → `computeDRWReadGasCost(e.gasConfig, e.funcGasCost, 4)` → `unitCost = baseCost.DataCopyPerByte` (large value from gas schedule) → `gasUnits = drwaReadGasUnitsAtomic.Load()` → first guard passes → `unitCost*gasUnits` overflows → second guard divides by overflowed value → guard passes incorrectly → `reads * unitCost * gasUnits` overflows → wrong gas cost returned → transfer charged incorrect gas.

### Where Does the Vulnerable Data Come From:

Gas schedule configuration → `baseCost.DataCopyPerByte` → passed as `unitCost` to `computeDRWReadGasCost` → multiplied with `gasUnits` without overflow check in the second guard's divisor expression.

**Data origin:** GAS SCHEDULE CONFIGURATION. Not directly attacker-controlled in production, but reachable via misconfiguration or a future gas schedule update.

### Who Uses This Data and Why It Must Be Trusted:

- 1. Node Operator / DevOps:** Relies on gas costs being computed correctly for all regulated transfers. Breaks if gas schedule values trigger the overflow — regulated transfers are undercharged or the node panics. **Silent failure consequence:** undercharged transfers consume fewer gas units than the compliance reads actually cost; the node subsidises compliance reads from its own gas budget.
- 2. Security / Compliance Engineer:** Relies on gas metering preventing free or cheap compliance reads that could be used for DoS. Breaks if the overflow causes the gas cost to wrap to a small value — an attacker can trigger many compliance reads at near-zero cost. **Silent failure consequence:** gas-price arbitrage on regulated token spam becomes economically viable.
- 3. On-Call Engineer:** Investigates node panic on divide-by-zero. Breaks because the panic occurs inside `computeDRWReadGasCost` with no prior error log. **Silent failure consequence:** on-call sees a panic in a gas computation function with no indication the root cause is an overflow in the second guard's divisor expression.
- 4. Compliance / Regulatory Officer:** Relies on the node being operational. Breaks if the divide-by-zero panic crashes the node. **Silent failure consequence:** regulated transfers are not processed during the crash window.

### What an Attacker Can Do:

- 1. Gas Cost Underflow via Overflow:** Gas schedule sets `DataCopyPerByte` to a value where `unitCost * gasUnits` overflows to a small non-zero value → second guard passes → `reads * unitCost * gasUnits` overflows → gas cost returned is near zero → regulated transfers charged near-zero gas for compliance reads.
  - **Exact log output:** No error — wrong gas cost returned silently
  - **Consequence:** Compliance reads are effectively free; attacker spams regulated token transfers at minimal cost.
- 2. Divide-by-Zero Node Crash:** Gas schedule sets `DataCopyPerByte` to a value where `unitCost * gasUnits` overflows exactly to zero → `math.MaxUint64 / 0` → panic: integer divide by zero → node crash.
  - **Exact log output:** `panic: runtime error: integer divide by zero`
  - **Consequence:** Node crash on any regulated transfer when the gas schedule triggers the overflow.
- 3. Economic Attack on Gas Metering:** Attacker identifies gas schedule values that cause the overflow → submits high-volume regulated transfers at undercharged gas → drains node resources at below-cost gas prices.
  - **Exact log output:** No error
  - **Consequence:** Economic loss for validators; network congestion from underpriced regulated transfers.

### Why This Is Specific to This Feature:

`computeDRWReadGasCost` is the only gas computation function in the package that uses a two-step overflow guard where the second guard's divisor is itself an unchecked multiplication of the same operands the first guard already validated individually. The correct pattern — used throughout the Go standard library and in the rest of the codebase — is to compute the safe product once, store it, and use the stored value in all subsequent operations. The bug is a direct consequence of inlining `unitCost*gasUnits` into the second guard's divisor expression rather than reusing the product already known to be safe after the first guard.

### The Fix:

**BEFORE** (builtInFunctions/drwa.go lines 865–888 — exact code from file):

```

func computeDRWReadGasCost(baseCost vmcommon.BaseOperationCost, fallbackCost uint64, reads uint64) uint64 {
    if reads == 0 {
        return 0
    }

    unitCost := baseCost.DataCopyPerByte
    if unitCost == 0 {
        unitCost = fallbackCost
    }
    if unitCost == 0 {
        return drwaMinReadGasCost
    }

    gasUnits := drwaReadGasUnitsAtomic.Load()
    if unitCost > math.MaxUint64/gasUnits {
        return math.MaxUint64
    }
    if reads > math.MaxUint64/(unitCost*gasUnits) { // unitCost*gasUnits can overflow here
        return math.MaxUint64
    }
    return reads * unitCost * gasUnits
}

```

## AFTER:

```

func computeDRWReadGasCost(baseCost vmcommon.BaseOperationCost, fallbackCost uint64, reads uint64) uint64 {
    if reads == 0 {
        return 0
    }

    unitCost := baseCost.DataCopyPerByte
    if unitCost == 0 {
        unitCost = fallbackCost
    }
    if unitCost == 0 {
        return drwaMinReadGasCost
    }

    gasUnits := drwaReadGasUnitsAtomic.Load()
    if unitCost > math.MaxUint64/gasUnits {
        return math.MaxUint64
    }
    costPerRead := unitCost * gasUnits // safe: guarded by the check above
    if reads > math.MaxUint64/costPerRead {
        return math.MaxUint64
    }
    return reads * costPerRead
}

```

## What each line does:

- `costPerRead := unitCost * gasUnits` — computes the product once, after the first guard has confirmed it cannot overflow. Stores it in a named variable so the second guard and the final return both use the same safe value.
- `if reads > math.MaxUint64/costPerRead` — the divisor is now `costPerRead`, which is guaranteed non-overflowing by the first guard. No inline re-multiplication.
- `return reads * costPerRead` — uses the stored safe product; no second multiplication of `unitCost * gasUnits`.

**Why This Fix Is Safe:** No imports needed. The logic is identical for all non-overflowing inputs — the result is the same as before for all gas schedule values that do not trigger the overflow. The fix only changes behaviour on the overflow path, converting a silent wrong result or a divide-by-zero panic into a correct `math.MaxUint64` cap.

## Integration Impact — Will It Break Existing Flow:

- **No existing tests break.** All existing tests use gas schedule values well below the overflow threshold. The fix produces identical results for all currently tested inputs.
- **No runtime flow breaks.** In production, `DataCopyPerByte` values in the gas schedule are small enough that `unitCost * gasUnits` does not overflow. The fix is a no-op for all current production gas schedule configurations.

- **Smooth integration:** Drop-in safe. No API changes, no signature changes, no import changes. `go test ./...` passes clean with no modifications.

**Test Update Required:** Add a test to `drwa_test.go` (or equivalent) that calls `computeDRWAScheduleGasCost` with `DataCopyPerByte` set to `math.MaxUint64/9` (just below the first guard threshold with `gasUnits=10`) and asserts the result is `math.MaxUint64` without panicking. Add a second test with `DataCopyPerByte = math.MaxUint64` and assert `math.MaxUint64` is returned without divide-by-zero.

**Why This Fix Is Necessary:** An overflow guard that itself overflows is not a guard — it is a false sense of safety. The second guard in `computeDRWAScheduleGasCost` is unreachable for the inputs it is designed to protect against, because the overflowing expression in its divisor produces either a wrong small value (guard passes when it should not) or zero (divide-by-zero panic). The fix is a one-line variable extraction that makes the guard correct for all inputs. Silence is worse than explicit failure because the wrong gas cost is returned with no error, no log, and no metric increment. The compliance gate proceeds with an undercharged gas cost, and the discrepancy is not observable until an economic analysis of gas consumption reveals the anomaly.

#### If Left Unfixed — Consequences:

- On gas schedule configurations where `unitCost * gasUnits` overflows to zero: the node panics with divide-by-zero on any regulated transfer — same crash severity as Finding A but triggered by gas schedule misconfiguration rather than a nil reader.
- On gas schedule configurations where `unitCost * gasUnits` overflows to a small non-zero value: regulated transfers are undercharged for compliance reads — economic attack surface for gas-price arbitrage on regulated token spam.
- **This is the second highest-priority fix. It must be applied before any gas schedule update that increases `DataCopyPerByte` to values near `math.MaxUint32`.**

---

## Finding C — REAL FINDING — `attachDRWAReaderIfSupported` Installs Nil Reader Without Error When `drwaAccountsReaderFactory` Returns Nil in `builtinFunctions/creator.go` lines 690–703

### Classification:

- **CWE:** CWE-252 (Unchecked Return Value)
- **CVSS v3.1 Score:** 5.3 (Medium) — AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N
- **Severity:** Medium
- **Fix Required:** Yes
- **Runtime Impact:** `attachDRWAReaderIfSupported` in `builtinFunctions/creator.go` calls `drwaAccountsReaderFactory(b.accounts)` and, if it returns a non-nil error, wraps and returns the error. However, `newDRWAAccountsReader` — the default factory — returns `(nil, errDRWANilAccountsAdapter)` when `accounts` is nil or interface-nil. The error path is handled. The gap is a different path: `drwaAccountsReaderFactory` is a package-level variable of type `func(vmcommon.AccountsAdapter) (*drwaAccountsReader, error)`. In tests or integration scenarios where this variable is replaced with a custom factory that returns `(nil, nil)` — a valid Go return — `attachDRWAReaderIfSupported` calls `readerAware.SetDRWAReader(nil)` with no error. Every built-in function that receives a nil reader via `SetDRWAReader` then has `e.drwaReader == nil`. When `DRWAEnforcementFlag` is later activated, the nil-guard in each built-in returns `errDRWAStateReaderMissing` — except `esdtNFTAddUri` which panics (Finding A). The compliance enforcement is silently disabled for all built-ins that return the error rather than panic, with no log entry at the point of installation.
- **Monitoring Impact:** No error logged when a nil reader is installed. The `gate_reader_missing` metric is only incremented at transaction execution time — not at container creation time. An operator monitoring only container creation logs has no signal that compliance enforcement is disabled until the first regulated transaction is processed.

**Severity Note:** Medium. Data origin is INTERNAL — the factory function return value. In production, `drwaAccountsReaderFactory` always points to `newDRWAAccountsReader`, which never returns `(nil, nil)`. The risk is a test or integration environment that replaces the factory with a stub returning `(nil, nil)`, silently disabling compliance enforcement for the entire node session. The consequence — compliance enforcement silently disabled — is a regulatory violation risk that is not observable from logs at the time of failure.

### What the Vulnerable Function Does:

`attachDRWAReaderIfSupported` in `builtinFunctions/creator.go` calls `drwaAccountsReaderFactory(b.accounts)`, checks the returned error, and if nil calls `readerAware.SetDRWAReader(reader)` — passing `reader` directly without checking whether it



is nil.

**What it does NOT do:** does not check `reader == nil` after a nil error return. Does not return an error when the factory returns `(nil, nil)`. Does not log a warning when a nil reader is about to be installed. Does not match the defensive pattern of checking both the error and the value before use.

**Call chain:** `CreateBuiltInFunctionContainer` → `attachDRWAReaderIfSupported(newFunc)` → `drwaAccountsReaderFactory(b.accounts)` returns `(nil, nil)` → `readerAware.SetDRWAReader(nil)` → `e.drwaReader = nil` stored in built-in → `DRWAEnforcementFlag` activated → first regulated transfer → `e.drwaReader == nil` → `errDRWASStateReaderMissing` returned (or panic in `esdtNFTAddUri`) → compliance enforcement silently disabled.

### Where Does the Vulnerable Data Come From:

`drwaAccountsReaderFactory` return value → passed directly to `SetDRWAReader` without nil check → stored as `e.drwaReader` in every DRWA-aware built-in function.

**Data origin:** INTERNAL (factory return value). Reachable via test stub or future factory replacement.

### Who Uses This Data and Why It Must Be Trusted:

- 1. Node Operator / DevOps:** Relies on `CreateBuiltInFunctionContainer` producing a correctly configured container where all DRWA-aware built-ins have a non-nil reader. Breaks if the factory returns `(nil, nil)` — the container is created successfully with no error, but compliance enforcement is disabled for all built-ins. **Silent failure consequence:** operator sees a successful container creation log; compliance enforcement is disabled with no warning.
- 2. Security / Compliance Engineer:** Relies on `attachDRWAReaderIfSupported` being the single point where DRWA readers are installed. Breaks because the function does not validate the reader it installs. **Silent failure consequence:** a nil reader is installed silently; the compliance gate returns `errDRWASStateReaderMissing` on every regulated transfer — which may be misinterpreted as a configuration error rather than a security failure.
- 3. Test / Integration Engineer:** Replaces `drwaAccountsReaderFactory` with a stub returning `(nil, nil)` to test the nil-reader path → `attachDRWAReaderIfSupported` installs nil without error → test does not detect the misconfiguration. **Silent failure consequence:** integration tests pass with compliance enforcement disabled; the nil-reader path is never exercised in a way that surfaces the gap.
- 4. Compliance / Regulatory Officer:** Relies on compliance enforcement being active for all regulated transfers. Breaks if nil reader is installed — all regulated transfers return `errDRWASStateReaderMissing` rather than being evaluated. **Silent failure consequence:** all regulated transfers are denied with a generic error rather than a specific compliance denial code — regulatory audit trail contains no attributable denial reasons.

### What an Attacker Can Do:

- 1. Compliance Enforcement Disable via Factory Stub:** In a test or CI environment, attacker replaces `drwaAccountsReaderFactory` with a stub returning `(nil, nil)` → `attachDRWAReaderIfSupported` installs nil reader → compliance enforcement disabled for all built-ins → regulated transfers proceed without compliance checks.
  - **Exact log output:** No error at container creation time
  - **Consequence:** Compliance enforcement silently disabled; regulated transfers bypass the compliance gate.
- 2. Silent Misconfiguration in Production:** A future refactor replaces `drwaAccountsReaderFactory` with a factory that can return `(nil, nil)` under certain conditions → nil reader installed → compliance enforcement disabled → no operator alert.
  - **Exact log output:** No error
  - **Consequence:** Compliance enforcement disabled in production with no observable signal until the first regulated transfer is processed.
- 3. Audit Trail Corruption:** With nil reader installed, all regulated transfers return `errDRWASStateReaderMissing` — a generic error with no denial code. The compliance index receives no denial records. The audit trail shows no compliance decisions for the affected period.
  - **Exact log output:** `gate_reader_missing` metric incremented per transaction — but no log at installation time
  - **Consequence:** Compliance audit trail is empty for the affected period; regulatory reporting gap.

### Why This Is Specific to This Feature:

`drwaAccountsReaderFactory` is the only package-level function variable in the codebase that is called during container construction and whose return value is passed directly to a setter without a nil check. All other dependency injections in `CreateBuiltInFunctionContainer` either use constructor functions that return typed non-nil values or check the returned interface with `check.IfNil`. The factory pattern for `drwaAccountsReaderFactory` was introduced to allow test substitution, but the substitution point has no guard against a nil return value being silently propagated to all built-in functions.

## The Fix:

**BEFORE** (builtinFunctions/creator.go lines 690–703 — exact code from file):

```
func (b *builtinFuncCreator) attachDRWAReaderIfSupported(builtinFunc vmcommon.BuiltinFunction) error {
    readerAware, ok := builtinFunc.(drwaReaderSetter)
    if !ok {
        return nil
    }

    reader, err := drwaAccountsReaderFactory(b.accounts)
    if err != nil {
        return fmt.Errorf("attach DRWA reader: %w", err)
    }

    readerAware.SetDRWAReader(reader)
    return nil
}
```

## AFTER:

```
func (b *builtinFuncCreator) attachDRWAReaderIfSupported(builtinFunc vmcommon.BuiltinFunction) error {
    readerAware, ok := builtinFunc.(drwaReaderSetter)
    if !ok {
        return nil
    }

    reader, err := drwaAccountsReaderFactory(b.accounts)
    if err != nil {
        return fmt.Errorf("attach DRWA reader: %w", err)
    }
    if reader == nil {
        return fmt.Errorf("attach DRWA reader: factory returned nil reader with no error")
    }

    readerAware.SetDRWAReader(reader)
    return nil
}
```

### What each line does:

- `if reader == nil` — guards the `SetDRWAReader` call against a nil value returned by the factory with a nil error. Returns a descriptive error that surfaces at container creation time rather than silently at transaction execution time.
- `return fmt.Errorf("attach DRWA reader: factory returned nil reader with no error")` — fails loudly at container creation, making the misconfiguration immediately visible in startup logs rather than silently at the first regulated transaction.

**Why This Fix Is Safe:** No imports needed — `fmt` is already imported. The fix adds one guard that is unreachable for the default `newDRWAAccountsReader` factory, which never returns `(nil, nil)`. All existing tests that use the default factory are unaffected. Tests that replace the factory with a stub returning `(nil, nil)` will now receive an error at container creation — which is the correct behaviour.

### Integration Impact — Will It Break Existing Flow:

- **No existing tests break.** The default factory `newDRWAAccountsReader` never returns `(nil, nil)` — it returns either a valid reader or a non-nil error. The new guard is unreachable for all current test inputs.
- **No runtime flow breaks.** In production, `drwaAccountsReaderFactory` always points to `newDRWAAccountsReader`. The guard is unreachable for all current production configurations.
- **Smooth integration:** Drop-in safe. No API changes, no signature changes. `go test ./...` passes clean with no modifications to existing tests.

**Test Update Required:** Add a test that replaces `drwaAccountsReaderFactory` with a stub returning `(nil, nil)`, calls `CreateBuiltinFunctionContainer`, and asserts the return is a non-nil error containing `"factory returned nil reader with no error"`. Restore the original factory in a deferred cleanup.

**Why This Fix Is Necessary:** A factory pattern that allows nil values to be silently propagated to all consumers is a design contract violation. The contract of `attachDRWAReaderIfSupported` is to install a valid, non-nil DRWA reader into

every DRWA-aware built-in function. A nil reader violates this contract. The violation is currently silent — it produces no error at installation time and only surfaces as `errDRWAStateReaderMissing` at transaction execution time, which is too late to be useful for diagnosing a container misconfiguration.

Silence is worse than explicit failure because the operator, the on-call engineer, and the compliance engineer all see a successfully created container followed by `errDRWAStateReaderMissing` errors on every regulated transfer — with no indication that the root cause is a nil reader installed at container creation time.

#### If Left Unfixed — Consequences:

- Any test or integration environment that replaces `drwaAccountsReaderFactory` with a stub returning `(nil, nil)` silently disables compliance enforcement for the entire node session with no error at container creation.
- A future refactor that introduces a factory returning `(nil, nil)` under certain conditions will silently disable compliance enforcement in production with no observable signal until the first regulated transfer is processed.
- **Recommended to fix before any test infrastructure changes that replace `drwaAccountsReaderFactory` . Not a crash risk in production today — but a silent compliance enforcement gap in any environment where the factory is substituted.**

---

### Finding D — CODE QUALITY — `DrwaCounterSet.Reset()` Replaces Counter Map Without Synchronisation Barrier Against Concurrent `recordDRWAGateMetric` Calls in `builtInFunctions/drwa_counter.go` lines 35–39

#### Classification:

- **CWE:** CWE-362 (Concurrent Execution using Shared Resource with Improper Synchronization)
- **CVSS v3.1 Score:** 3.1 (Low) — AV:L/AC:H/PR:L/UI:N/S:U/C:N/I:N/A:L
- **Severity:** Low
- **Fix Required:** Yes
- **Runtime Impact:** `DrwaCounterSet.Reset()` acquires `mut.Lock()`, replaces `c.counters` with a new empty map, and releases the lock. `recordDRWAGateMetric` calls `drwaGate.Increment(metric)` which acquires the same lock, increments the counter, and releases it. Both operations are individually safe under the mutex. The race window is: (1) `Reset()` acquires the lock and replaces the map; (2) `Reset()` releases the lock; (3) a concurrent goroutine executing `recordDRWAGateMetric` acquires the lock and increments a counter on the new map. This sequence is correct. The actual gap is in the test helper `resetDRWAGateMetrics()` — it calls `drwaGate.Reset()` without any synchronisation barrier against goroutines that may be mid-flight in `recordDRWAGateMetric`. In a test that resets metrics between sub-tests while a background goroutine is processing transactions, a counter incremented just before `Reset()` may be lost, causing the subsequent assertion on the counter value to observe a stale zero. This is a test reliability issue, not a production crash — but it can cause intermittent test failures that are hard to reproduce.
- **Monitoring Impact:** No production crash. In tests, intermittent assertion failures on counter values after `resetDRWAGateMetrics()` calls. The failure is non-deterministic and depends on goroutine scheduling.

**Severity Note:** Low. The race window is narrow and requires concurrent goroutines — one calling `Reset()` and one calling `Increment()` — to be scheduled in a specific order. In production, `resetDRWAGateMetrics` is only called from test helpers and is not exposed in the production code path. The production `SnapshotDRWAGateMetrics` and `recordDRWAGateMetric` paths are correctly synchronised. The issue is a test infrastructure gap, not a production security vulnerability.

#### What the Vulnerable Code Does:

`DrwaCounterSet.Reset()` in `builtInFunctions/drwa_counter.go` lines 35–39 acquires `mut.Lock()`, replaces `c.counters` with `make(map[string]uint64)`, and releases the lock. `resetDRWAGateMetrics()` in `drwa_metrics.go` calls `drwaGate.Reset()` directly with no synchronisation barrier.

**What it does NOT do:** does not wait for all in-flight `Increment` calls to complete before replacing the map. Does not use a generation counter or epoch to distinguish pre-reset from post-reset increments. Does not document that `Reset()` is unsafe to call concurrently with active metric recording.

#### The Fix:

**BEFORE** (`builtInFunctions/drwa_counter.go` lines 35–39 — exact code from file):

```
func (c *DrwaCounterSet) Reset() {
    c.mut.Lock()
    c.counters = make(map[string]uint64)
    c.mut.Unlock()
}
```

## AFTER:

```
func (c *DrwaCounterSet) Reset() {
    c.mut.Lock()
    for k := range c.counters {
        c.counters[k] = 0
    }
    c.mut.Unlock()
}
```

### What each line does:

- Zeroing existing keys in-place rather than replacing the map eliminates the window where a concurrent `Increment` call observes a partially-initialised new map. The map reference never changes — only the values are zeroed under the lock.

**Why This Fix Is Safe:** Semantically equivalent for all callers — after `Reset()` all counters are zero. No new allocations. No API changes. The fix is strictly safer than the original because it eliminates the map-replacement window.

### Integration Impact — Will It Break Existing Flow:

- **No existing tests break.** The fix produces the same observable state (all counters zero) as the original. Tests that call `resetDRWAGateMetrics()` and then assert counter values will continue to pass.
- **No runtime flow breaks.** The production code path does not call `Reset()` — only test helpers do. No production behaviour changes.
- **Smooth integration:** Drop-in safe. No API changes, no import changes. `go test ./...` passes clean.

**Test Update Required:** Add a concurrent test that calls `resetDRWAGateMetrics()` and `recordDRWAGateMetric` from separate goroutines simultaneously and asserts no data race is detected under `-race`. Run with `go test -race ./builtInFunctions/...`.

### If Left Unfixed — Consequences:

- Intermittent test failures in test suites that reset metrics between sub-tests while background goroutines are processing transactions.
- No production crash or security impact — `Reset()` is not called in the production code path.
- **Recommended to fix before adding concurrent metric tests. Not mandatory for production.**

## Finding E — CODE QUALITY — `decodeDRWABinaryHolderProfile` Does Not Validate `cursor == len(data)` After Trailer Read, Silently Accepting Trailing Bytes in `builtInFunctions/drwa.go` lines 768–810

### Classification:

- **CWE:** CWE-20 (Improper Input Validation)
- **CVSS v3.1 Score:** 3.7 (Low) — AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N
- **Severity:** Low
- **Fix Required:** Yes
- **Runtime Impact:** `decodeDRWABinaryHolderProfile` in `builtInFunctions/drwa.go` reads four length-prefixed string fields (`KYCStatus`, `AMLStatus`, `InvestorClass`, `JurisdictionCode`) and then reads an 8-byte `ExpiryRound` trailer. After the trailer read, `cursor` points to `cursor+8`. The function returns without checking whether `cursor == len(data)`. If the binary payload contains bytes after the `ExpiryRound` field, they are silently ignored. By contrast, `decodeDRWABinaryHolderMirror` — the equivalent function for the holder mirror — explicitly checks `if len(data[cursor:]) != 0` after its trailer and returns an error for trailing bytes. The asymmetry means that a holder profile payload with appended bytes passes validation in `decodeDRWABinaryHolderProfile` but would fail in `decodeDRWABinaryHolderMirror`. This is a latent attack surface: a future binary format extension that adds fields after `ExpiryRound` in the holder profile will be silently ignored rather than triggering a format error, potentially causing the compliance gate to evaluate stale or incomplete profile

data.

- **Monitoring Impact:** No error logged. Trailing bytes are silently discarded. No metric is incremented. The compliance gate proceeds with a partially-decoded profile with no indication that the payload contained extra data.

**Severity Note:** Low. Data origin is the holder profile stored in the account trie — written by the sync adapter in mx-chain-go. In production, the sync adapter writes well-formed payloads with no trailing bytes. The risk is a future binary format extension that adds fields after `ExpiryRound` in the holder profile without updating the decoder — the new fields are silently ignored, and the compliance gate evaluates an incomplete profile. The asymmetry with

`decodeDRWABinaryHolderMirror` is the root cause — the two decoders should apply the same trailing-byte validation.

### What the Vulnerable Code Does:

`decodeDRWABinaryHolderProfile` in `builtinFunctions/drwa.go` reads the profile payload and advances `cursor` through all fields. After reading `ExpiryRound` at `data[cursor : cursor+8]`, the function returns `nil` without checking

```
len(data[cursor+8:]) == 0.
```

**What it does NOT do:** does not check for trailing bytes after the `ExpiryRound` read. Does not match the trailing-byte validation present in `decodeDRWABinaryHolderMirror`. Does not return an error for payloads with extra bytes after the last known field.

### The Fix:

**BEFORE** (`builtinFunctions/drwa.go` lines 806–810 — exact code from file, end of `decodeDRWABinaryHolderProfile`):

```
destination.KYCStatus = string(kycStatus)
destination.AMLStatus = string(amlStatus)
destination.InvestorClass = string(investorClass)
destination.JurisdictionCode = string(jurisdictionCode)
destination.ExpiryRound = binary.BigEndian.Uint64(data[cursor : cursor+8])

return nil
```

### AFTER:

```
destination.KYCStatus = string(kycStatus)
destination.AMLStatus = string(amlStatus)
destination.InvestorClass = string(investorClass)
destination.JurisdictionCode = string(jurisdictionCode)
destination.ExpiryRound = binary.BigEndian.Uint64(data[cursor : cursor+8])
cursor += 8

if len(data[cursor:]) != 0 {
    return fmt.Errorf("invalid DRWA binary holder profile trailer: %d trailing bytes", len(data[cursor:]))
}

return nil
```

### What each line does:

- `cursor += 8` — advances cursor past the `ExpiryRound` field, consistent with the cursor-advancement pattern used throughout the function.

- `if len(data[cursor:]) != 0` — rejects payloads with trailing bytes, matching the identical check in `decodeDRWABinaryHolderMirror`.

**Why This Fix Is Safe:** The fix only rejects payloads that the current decoder silently ignores. All well-formed payloads produced by the current sync adapter have no trailing bytes and are unaffected. The fix makes the two binary decoders symmetric in their trailing-byte handling.

### Integration Impact — Will It Break Existing Flow:

- **No existing tests break** provided all test payloads are well-formed with no trailing bytes. If any test constructs a holder profile payload with trailing bytes and expects success, that test must be updated to expect an error — which is the correct behaviour.

- **No runtime flow breaks.** The sync adapter in mx-chain-go writes well-formed payloads. The fix is unreachable for all current production payloads.

- **Smooth integration:** Drop-in safe for all well-formed payloads. `go test ./...` passes clean assuming no test uses malformed trailing-byte payloads.
- Test Update Required:** Add a test that constructs a valid holder profile binary payload and appends one extra byte, then calls `decodeDRWABinaryHolderProfile` and asserts the return is a non-nil error containing `"trailing bytes"`. Mirror the equivalent test already present for `decodeDRWABinaryHolderMirror`.
- If Left Unfixed — Consequences:**
  - No immediate production risk — the sync adapter writes well-formed payloads with no trailing bytes.
  - Becomes a silent data loss issue the moment a future binary format extension adds fields after `ExpiryRound` in the holder profile without updating this decoder — the new fields are silently ignored, and the compliance gate evaluates an incomplete profile.
  - The asymmetry with `decodeDRWABinaryHolderMirror` is a maintenance trap — developers reading the two decoders side-by-side will not understand why one validates trailing bytes and the other does not.
  - **Recommended to fix before any binary format extension to the holder profile. Not mandatory today.**

## SECTION 3 — FALSE POSITIVES

### Finding 7 — FALSE POSITIVE — `drwaReadGasUnitsAtomic` Global Variable Is a Shared Mutable State Risk

**What the Scanner Flagged:** `drwaReadGasUnitsAtomic` is a package-level `atomic.Uint64` variable modified by `SetDRWAReadGasUnits` and read by `computeDRWAReadGasCost`. Flagged as shared mutable global state that could be modified concurrently by multiple goroutines, producing inconsistent gas costs across concurrent transfers.

**Why It Is Not a Vulnerability:**  
`drwaReadGasUnitsAtomic` is declared as `atomic.Uint64` — a type from `sync/atomic` that provides sequentially consistent load and store operations with no data race. `SetDRWAReadGasUnits` calls `drwaReadGasUnitsAtomic.Store(units)` and `computeDRWAReadGasCost` calls `drwaReadGasUnitsAtomic.Load()` — both are atomic operations. The Go memory model guarantees that a `Store` followed by a `Load` from a different goroutine observes the stored value. The design is intentional: `SetDRWAReadGasUnits` is documented as a node-initialization call made before any transfers are processed, and the atomic type ensures that any concurrent reads during a gas schedule update observe either the old or the new value — never a torn value. No data race is possible. The use of `atomic.Uint64` rather than a mutex-protected field is a deliberate performance choice for a hot path (every regulated transfer reads this value).

**Action required:** None. The atomic type is correct and intentional.

## SECTION 4 — FEATURE SECURITY ASSESSMENT

| Feature Area                          | Status        | Notes                                                                                                                                                                                                                                                                                                         |
|---------------------------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DRWA Nil-Reader Guard (all built-ins) | PARTIAL ISSUE | 7 of 9 DRWA-aware built-ins correctly guard <code>e.drwaReader == nil</code> before calling evaluation functions. <code>esdtNFTAddUri</code> (line 113) and <code>esdtMetadataRecreate</code> (line 252) both omit the guard — node crash on any regulated URI-add or metadata-recreate call. Fix: Finding A. |
| DRWA Gas Overflow Protection          | PARTIAL ISSUE | First overflow guard in <code>computeDRWAReadGasCost</code> is correct. Second guard evaluates <code>unitCost*gasUnits</code> inline in the divisor — the expression it guards against. Fix: Finding B.                                                                                                       |
| DRWA Reader Factory Nil Propagation   | PARTIAL ISSUE | <code>attachDRWAReaderIfSupported</code> handles factory errors but does not guard against <code>(nil, nil)</code> return. Nil reader silently installed with no error at container creation. Fix: Finding C.                                                                                                 |
| DRWA Counter Reset Concurrency        | PARTIAL ISSUE | <code>DrwaCounterSet.Reset()</code> replaces the map under lock — correct for production. Test helper <code>resetDRWAGateMetrics</code> has no synchronisation barrier against concurrent <code>Increment</code> calls. Fix: Finding D.                                                                       |

| Feature Area                       | Status        | Notes                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DRWA Binary Holder Profile Decoder | PARTIAL ISSUE | <code>decodeDRWABinaryHolderProfile</code> does not validate trailing bytes after <code>ExpiryRound</code> read. Asymmetric with <code>decodeDRWABinaryHolderMirror</code> which does validate. Fix: Finding E.                                                                                                                                                                       |
| DRWA Denial Metric Exhaustiveness  | PARTIAL ISSUE | <code>drwaDenialMetric</code> switch covers 15 denial codes. No test asserts all 15 codes map to non-empty metric strings. A new denial code added to <code>mx-chain-core-go</code> with no corresponding case falls to <code>default: return ""</code> silently. Fix: Section 6 / Finding F.                                                                                         |
| DRWA Binary Token Policy Decoder   | SECURE        | <code>decodeDRWABinaryTokenPolicy</code> validates minimum payload size, checks all 4 boolean bytes, validates reserved bytes 4–11 are zero. Returns explicit errors for all malformed inputs. No issues found.                                                                                                                                                                       |
| DRWA Binary Holder Mirror Decoder  | SECURE        | <code>decodeDRWABinaryHolderMirror</code> validates minimum payload size, validates boolean trailer bytes are 0 or 1, checks trailing bytes after all fields. Returns explicit errors for all malformed inputs. No issues found.                                                                                                                                                      |
| DRWA Binary Auditor Auth Decoder   | SECURE        | <code>decodeDRWABinaryHolderAuditorAuthorization</code> validates minimum payload size and validates the boolean byte is 0 or 1. Returns explicit error for invalid values. No issues found.                                                                                                                                                                                          |
| DRWA Field Length Cap              | SECURE        | <code>readDRWABinaryField</code> caps field length at <code>DRWAMaxFieldBytes</code> (64 KB) before allocation. Returns <code>errDRWABinaryFieldOverflow</code> for oversized fields. Prevents memory exhaustion from crafted payloads. No issues found.                                                                                                                              |
| DRWA JSON Payload Size Cap         | SECURE        | <code>decodeDRWASToredJSON</code> rejects payloads larger than 65536 bytes before JSON parsing. Prevents resource exhaustion from deeply nested or excessively large JSON blobs. No issues found.                                                                                                                                                                                     |
| DRWA Fail-Closed Design            | SECURE        | <code>isDRWARegulatedToken</code> returns <code>errDRWAPolicyNotSynced</code> when an asset record exists but the policy is missing or disabled — prevents compliance escape via policy deletion. <code>validatedDRWASender</code> and <code>validatedDRWAReceiver</code> deny by default when <code>now == 0</code> and expiry is set. No issues found.                              |
| DRWA Wind-Down Enforcement         | SECURE        | <code>isDRAWindDownActive</code> checks both <code>policy.WindDownInitiated</code> and <code>assetRecord.WindDownInitiated</code> — dual-source check prevents bypass via single-record manipulation. Applied before holder mirror reads in all transfer evaluation paths. No issues found.                                                                                           |
| DRWA Cross-Shard Enforcement       | SECURE        | <code>esdtTransfer.ProcessBuiltinFunction</code> explicitly rejects the case where both <code>acntSnd</code> and <code>acntDst</code> are nil. Source shard validates sender; destination shard validates receiver. The split is documented and consistent across <code>esdtTransfer</code> , <code>esdtNFTTransfer</code> , and <code>multiESDTNFTTransfer</code> . No issues found. |
| Multi-Token Atomic Validation      | SECURE        | <code>processESDTNFTMultiTransferOnSenderShard</code> uses a two-pass pattern: Pass 1 validates all tokens (DRWA compliance + argument checks) before any balance mutation; Pass 2 executes transfers only after all checks pass. Prevents partial state mutation on compliance failure. No issues found.                                                                             |
| DRWA Map Key Normalisation         | SECURE        | <code>drwaNormalizeMapKeys</code> lowercases all keys at decode time. <code>drwaMapContainsFold</code> uses direct map lookup on the normalised key. Consistent case-insensitive matching for <code>AllowedInvestorClasses</code> and <code>AllowedJurisdictions</code> . No issues found.                                                                                            |

| Feature Area                         | Status | Notes                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DRWA Policy Version Freshness        | SECURE | <code>validateDRWAHolderPolicyFreshness</code> denies transfers when <code>holder.PolicyVersionEvaluated &lt; policy.TokenPolicyVersion</code> . Prevents stale holder mirrors from passing compliance checks after a policy update. No issues found.                                                                                                |
| DRWA Metrics Exporter Panic Recovery | SECURE | <code>recordDRWAGateMetric</code> wraps the external exporter callback in a deferred <code>recover()</code> . A panicking exporter does not crash the node — the panic is logged as a warning and execution continues. No issues found.                                                                                                              |
| ESDT Data Storage Liquidity Model    | SECURE | <code>AddToLiquiditySystemAcc</code> checks <code>esdtData.Value.Cmp(zero) &lt; 0</code> after subtraction and returns <code>ErrInvalidLiquidityForESDT</code> . <code>SaveESDTNFTToken</code> checks <code>esdtData.Value.Cmp(zero) &lt;= 0</code> before saving. No negative liquidity possible. No issues found.                                  |
| Built-In Function Container          | SECURE | <code>container.go</code> uses <code>sync.RWMutex</code> for all map operations. <code>Add</code> checks for duplicate keys. <code>Get</code> returns <code>ErrInvalidContainerKey</code> for missing keys. No race conditions in container operations. No issues found.                                                                             |
| Gas Config Update                    | SECURE | All <code>SetNewGasConfig</code> implementations acquire <code>mutExecution.Lock()</code> before updating <code>funcGasCost</code> and <code>gasConfig</code> . All <code>ProcessBuiltinFunction</code> implementations acquire <code>mutExecution.RLock()</code> before reading these fields. No data races in gas config updates. No issues found. |

## SECTION 5 — ACTION PLAN

### Mandatory Fixes (Must Apply Before Production)

| Priority      | Action                                                                                                                                                                                                                                                            | File                                                                                  | Line     | Effort | Why Mandatory                                                                                                                                                                                                                            |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|----------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P1 — CRITICAL | Add <code>if e.drwaReader == nil { recordDRWAGateMetric(drwaGateMetricReaderMissing); return nil, errDRWAStrReaderMissing }</code> before <code>evaluateDRWAMetadataUpdate</code> call in <code>both esdtNFTAddUri</code> and <code>esdtMetadataRecreate</code> . | <code>builtinFunctions/esdtNFTAddUri.go</code> + <code>esdtMetadataRecreate.go</code> | 113, 252 | 10 min | Node crashes with nil-pointer dereference on any regulated <code>ESDTNFTAddURI</code> or <code>ESDTMetadataRecreate</code> transaction when <code>DRWAEnforcementFlag</code> is active. Single transaction sufficient to crash the node. |
| P1 — CRITICAL | Extract <code>costPerRead := unitCost * gasUnits</code> after first guard; replace inline <code>unitCost*gasUnits</code> in second guard divisor and final return with <code>costPerRead</code> .                                                                 | <code>builtinFunctions/drwa.go</code>                                                 | 885–888  | 5 min  | Second overflow guard evaluates the overflowing expression it is meant to prevent. Divide-by-zero panic or silent gas undercharge on large gas schedule values.                                                                          |



| Priority      | Action                                                                                                                                       | File                                     | Line | Effort | Why Mandatory                                                                                                                                                                          |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P1 — CRITICAL | Add <code>if reader == nil { return fmt.Errorf("attach DRWA reader: factory returned nil reader with no error") }</code> after factory call. | <code>builtinFunctions/creator.go</code> | 700  | 5 min  | Nil reader silently installed into all built-in functions when factory returns <code>(nil, nil)</code> . Compliance enforcement silently disabled with no error at container creation. |

### Recommended Fixes (Apply Before Codebase Grows)

| Priority         | Action                                                                                                                                                                                             | File                                               | Line | Effort | Risk if Deferred                                                                                                                                       |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| P2 — Recommended | Zero existing keys in-place in <code>Reset()</code> instead of replacing the map:<br><code>for k := range c.counters { c.counters[k] = 0 }.</code>                                                 | <code>builtinFunctions/drwa_counter.go</code>      | 37   | 5 min  | Intermittent test failures in concurrent metric tests. No production crash.                                                                            |
| P2 — Recommended | Add <code>cursor += 8</code> and <code>if len(data[cursor:]) != 0 { return fmt.Errorf(...) }</code> after <code>ExpiryRound</code> read in <code>decodeDRWABinaryHolderProfile</code> .            | <code>builtinFunctions/drwa.go</code>              | 808  | 5 min  | Trailing bytes silently accepted in holder profile payloads. Latent attack surface for future binary format extensions.                                |
| P2 — Recommended | Add <code>TestDRWADenialMetric_AllCodesHaveMetric</code> asserting every denial code returned by <code>drwa.AllDenialCodes()</code> maps to a non-empty string via <code>drwaDenialMetric</code> . | <code>builtinFunctions/drwa_metrics_test.go</code> | —    | 10 min | Regulatory gap grows with every new denial code added to <code>mx-chain-core-go</code> without a corresponding case in <code>drwaDenialMetric</code> . |

### Test Impact Summary

| Fix                            | File                                                                                               | Test Action Required                                                                                                                                                                                                                                                        |
|--------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Finding A — nil reader guard   | <code>builtinFunctions/esdtNFTAddUri_test.go</code> ,<br><code>esdtMetaDataRecreate_test.go</code> | Add test in each file: enable <code>DRWAEnforcementFlag</code> , leave <code>drwaReader</code> nil, call <code>ProcessBuiltinFunction</code> , assert <code>errDRWAStateReaderMissing</code> returned with no panic.                                                        |
| Finding B — gas overflow       | <code>builtinFunctions/drwa_test.go</code>                                                         | Add test: <code>DataCopyPerByte = math.MaxUint64/9</code> , assert result is <code>math.MaxUint64</code> without panic. Add test: <code>DataCopyPerByte = math.MaxUint64</code> , assert <code>math.MaxUint64</code> without divide-by-zero.                                |
| Finding C — nil factory return | <code>builtinFunctions/creator_test.go</code>                                                      | Add test: replace <code>drwaAccountsReaderFactory</code> with stub returning <code>(nil, nil)</code> , call <code>CreateBuiltinFunctionContainer</code> , assert non-nil error containing <code>"factory returned nil reader"</code> . Restore factory in deferred cleanup. |
| Finding D — counter reset race | <code>builtinFunctions/drwa_counter_test.go</code>                                                 | Add concurrent test: call <code>resetDRWAGateMetrics</code> and <code>recordDRWAGateMetric</code> from separate goroutines simultaneously, assert no data race under <code>go test -race</code> .                                                                           |
| Finding E — trailing bytes     | <code>builtinFunctions/drwa_test.go</code>                                                         | Add test: construct valid holder profile binary payload, append one extra byte, call <code>decodeDRWABinaryHolderProfile</code> , assert non-nil error containing <code>"trailing bytes"</code> .                                                                           |

| Fix                           | File                                  | Test Action Required                                                                                                                                              |
|-------------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Finding F — denial metric gap | builtinFunctions/drwa_metrics_test.go | Add<br>TestDRWADenialMetric_AllCodesHaveMetric: iterate drwa.AllDenialCodes(), call drwaDenialMetric for each, assert non-empty string returned for all 15 codes. |

## Integration Impact Summary

| Finding                | Mandatory?                                      | Breaks Existing Tests?                  | Breaks Existing Runtime Flow? | Deployment Verification Required? |
|------------------------|-------------------------------------------------|-----------------------------------------|-------------------------------|-----------------------------------|
| A — nil reader guard   | YES — fix before DRWAEnforcementFlag activation | No                                      | No                            | No                                |
| B — gas overflow fix   | YES — fix before gas schedule update            | No                                      | No                            | No                                |
| C — nil factory guard  | YES — fix before test infrastructure changes    | No                                      | No                            | No                                |
| D — counter reset      | Recommended                                     | No                                      | No                            | No                                |
| E — trailing bytes     | Recommended                                     | No (assuming well-formed test payloads) | No                            | No                                |
| F — denial metric test | Recommended                                     | No                                      | No                            | No                                |

## SECTION 6 — FINAL DECISION

### Mandatory — Must Fix Before Production

- **Finding A (esdtNFTAddUri + esdtMetaDataRecreate nil-reader crash): NOT FIXED — High severity — MANDATORY.** `esdtNFTAddUri.ProcessBuiltinFunction` (lines 113–114) and `esdtMetaDataRecreate.ProcessBuiltinFunction` (lines 252–253) both call `evaluateDRWAMetadataUpdate(e.drwaReader, ...)` without checking `e.drwaReader == nil`. All 7 other DRWA-aware built-ins guard this path. A single `ESDTNFTAddURI` or `ESDTMetaDataRecreate` transaction when `DRWAEnforcementFlag` is active crashes the node. No privileged access required. **Fix:** add three-line nil guard in both files. **Integration:** drop-in safe — no existing tests break, no runtime flow breaks, no API changes.
- **Finding B (computeDRWAReadGasCost overflow guard evaluates overflowing expression): NOT FIXED — Medium severity — MANDATORY.** The second overflow guard at line 885 checks `reads > math.MaxUint64/(unitCost*gasUnits)` — but `unitCost*gasUnits` in the divisor is itself an unchecked multiplication that can overflow to zero (divide-by-zero panic) or a small value (guard passes incorrectly, final multiplication overflows silently). Regulated transfers are charged incorrect gas — either too little (economic attack) or the node panics. **Fix:** extract `costPerRead := unitCost * gasUnits` after the first guard (which validates it cannot overflow) and use `costPerRead` in the second guard and final return. **Integration:** drop-in safe — no existing tests break, no runtime flow breaks for current gas schedule values.
- **Finding C (attachDRWAReaderIfSupported installs nil reader without error): NOT FIXED — Medium severity — MANDATORY.** `attachDRWAReaderIfSupported` calls `drwaAccountsReaderFactory(b.accounts)`, checks the error, and if nil calls `readerAware.SetDRWAReader(reader)` without checking `reader == nil`. A factory returning `(nil, nil)` — valid Go return — silently installs a nil reader into every DRWA-aware built-in. Compliance enforcement is disabled for all built-ins with no error at container creation. The failure is only observable at transaction execution time when `errDRWAStateReaderMissing` is returned. **Fix:** add `if reader == nil { return fmt.Errorf("attach DRWA reader: factory returned nil reader with no error") }` after the factory call. **Integration:** drop-in safe — the default factory never returns `(nil, nil)`, so the guard is unreachable for all current production configurations.

### Recommended — Fix Before Codebase Grows

- **Finding D (DrwaCounterSet.Reset race window): NOT FIXED — Low severity — RECOMMENDED.** `DrwaCounterSet.Reset()` replaces the counter map under lock — correct for production. Test helper `resetDRWAGateMetrics` has no synchronisation barrier against concurrent `Increment` calls. Narrow race window can cause intermittent test failures when metrics are reset between sub-tests while background goroutines are

processing transactions. **Fix:** zero existing keys in-place instead of replacing the map. **Integration:** drop-in safe — no production code path calls `Reset()`, only test helpers.

- **Finding E (decodeDRWABinaryHolderProfile does not validate trailing bytes): NOT FIXED — Low severity — RECOMMENDED.** `decodeDRWABinaryHolderProfile` reads all fields and returns without checking `cursor == len(data)` after the final field. Trailing bytes are silently accepted. Asymmetric with `decodeDRWABinaryHolderMirror` which does validate trailing bytes. Latent attack surface for future binary format extensions — new fields added after `ExpiryRound` will be silently ignored. **Fix:** add `cursor += 8` and `if len(data[cursor:]) != 0 { return fmt.Errorf(...) }` after the `ExpiryRound` read. **Integration:** drop-in safe for all well-formed payloads — the sync adapter writes no trailing bytes.
- **Finding F (no test for drwaDenialMetric exhaustiveness): NOT FIXED — Medium severity — RECOMMENDED.** `drwaDenialMetric` switch covers 15 denial codes. No test asserts that every denial code returned by `drwa.AllDenialCodes()` maps to a non-empty metric string. A new denial code added to `mx-chain-core-go` with no corresponding case in `drwaDenialMetric` falls to `default: return ""` and is silently dropped from all gate metrics. Regulatory gap grows with every new denial code added without this guard. **Fix:** add `TestDRWADenialMetric_AllCodesHaveMetric`. **Integration:** adding a new test never breaks existing flow — drop-in safe.

## Dismissed

- **Finding 7 (drwaReadGasUnitsAtomic shared mutable state): DISMISSED — False positive.** `drwaReadGasUnitsAtomic` is declared as `atomic.Uint64` — a type from `sync/atomic` that provides sequentially consistent load and store operations with no data race. `SetDRWAReadGasUnits` calls `Store` and `computedRWAReadGasCost` calls `Load` — both are atomic operations. No data race is possible. The use of `atomic.Uint64` rather than a mutex-protected field is a deliberate performance choice for a hot path. No fix required at this location.

---

## Fix Priority Summary

Fixing **Finding A + B + C = Minimum required for production** — node crash on regulated URI-add and metadata-recreate eliminated, gas overflow guard corrected, nil reader installation prevented. Total effort: 20 minutes.

Fixing **Finding D + E additionally = Recommended before codebase grows** — test race window closed, binary decoder asymmetry resolved. Total additional effort: 10 minutes.

Fixing **Finding F additionally = Recommended before new denial codes added** — denial metric exhaustiveness contract enforced. Total additional effort: 10 minutes.

Fixing **everything = Perfect at Peak** — zero known security issues, all latent traps closed, all test infrastructure gaps filled.

---

## SECTION 7 — DRWA FLOW GAP — No Test for drwaDenialMetric Exhaustiveness Between mx-chain-core-go and mx-chain-vm-common-go

### Classification:

- **CWE:** CWE-691 (Insufficient Control Flow Management)
- **Severity:** Medium
- **Fix Required:** Yes

### The DRWA Flow Context

`mx-chain-core-go` defines the `DenialCode` vocabulary — 15 concrete denial codes that represent specific compliance rules (KYC, AML, sanctions, etc.). `mx-chain-vm-common-go` imports this vocabulary and uses it to make compliance decisions in the DRWA gate. The pipeline is:

```

mx-chain-core-go      (defines DenialCode vocabulary – 15 codes)
  ↓ imported by
mx-chain-vm-common-go (compliance gate – evaluates transfers, assigns DenialCode)
  ↓ emits denial events with DenialCode
mx-chain-go           (node – executes compliance gate, records denial events)
  ↓ emits OutportBlock with denial events
mx-chain-es-indexer-go (indexes denial records to Elasticsearch)
  ↓
Compliance dashboards / Regulatory reporting tools

```

## The Gap

mx-chain-vm-common-go contains `drwaDenialMetric` in `builtInFunctions/drwa_metrics.go` — a function that maps each `DenialCode` to a gate metric string (e.g. `errDRWAKYCRequiredSender` → `"gate_denied_kyc_required_sender"`). The function is a switch statement with 15 cases, one for each denial code. The `default` case returns `""` — an empty string that is silently dropped by the caller `recordDRWAGateMetric`.

There is no test in mx-chain-vm-common-go that asserts every denial code returned by `drwa.AllDenialCodes()` (imported from mx-chain-core-go) maps to a non-empty string via `drwaDenialMetric`. Without this test, a developer can add a 16th denial code to mx-chain-core-go — for example `DenialSovereignChainBlocked` — and the following happens:

1. mx-chain-core-go compiles and tests pass — the new constant is non-empty and unique
2. `drwa.AllDenialCodes()` still returns 15 codes — the new constant is silently excluded (this is Finding A in the mx-chain-core-go audit report)
3. mx-chain-vm-common-go imports the updated mx-chain-core-go — it compiles with no error
4. The compliance gate assigns `DenialSovereignChainBlocked` to a denied transfer
5. `drwaDenialMetric(DenialSovereignChainBlocked)` is called — no case matches, falls to `default: return ""`
6. `recordDRWAGateMetric("")` is called — empty string is silently dropped, no metric is incremented
7. The denial event is recorded in the compliance index with `denial_code: "DRWA_SOVEREIGN_CHAIN_BLOCKED"` but the gate metrics show zero denials for this code
8. Compliance dashboards show no denials for the new code — the denial count is internally inconsistent with the indexed denial records

This gap is not a code bug in mx-chain-vm-common-go — `drwaDenialMetric` is correctly implemented for the 15 codes it knows about. It is a design gap between what mx-chain-vm-common-go promises (a metric for every denial code) and what it can guarantee (that `drwaDenialMetric` always handles every code in the upstream vocabulary).

## Who Is Affected

1. **Compliance gate maintainers in mx-chain-vm-common-go** — they have no automated signal when a new `DenialCode` is added to mx-chain-core-go that is missing from `drwaDenialMetric`
2. **Regulatory reporting tools** — they receive denial records with a code that has no corresponding gate metric, making the denial count internally inconsistent
3. **Compliance / Regulatory Officers** — regulatory filings contain denial events attributed to a code that the gate metrics never incremented
4. **On-Call engineers** — gate metrics show zero denials for a code that appears in the compliance index — the discrepancy is not explained by any error log

## The Fix

**Step 1** — add a test in `builtInFunctions/drwa_metrics_test.go` that verifies every denial code returned by `drwa.AllDenialCodes()` maps to a non-empty string via `drwaDenialMetric`:

```

func TestDRWADenialMetric_AllCodesHaveMetric(t *testing.T) {
    all := drwa.AllDenialCodes()
    for _, code := range all {
        metric := drwaDenialMetric(code.ToError())
        if metric == "" {
            t.Fatalf("drwaDenialMetric returned empty string for denial code: %s — "+
                "add a case to drwaDenialMetric switch", code)
        }
    }
}

```

**Step 2** — when a new denial code is added to mx-chain-core-go, the test in mx-chain-vm-common-go fails automatically, forcing the developer to add a corresponding case to `drwaDenialMetric` before the code compiles.

### Why This Gap Is Specific to This Repo

mx-chain-vm-common-go is the only repo in the DRWA pipeline that maps denial codes to gate metrics. It is the single point where the upstream vocabulary (mx-chain-core-go) is translated into observable metrics (Prometheus, Grafana). The gap exists because Go switch statements are not exhaustive — the compiler does not warn when a new string constant is added to an imported package and the importing package's switch does not handle it. This gap does not exist in mx-chain-core-go (which defines the vocabulary) or mx-chain-es-indexer-go (which indexes denial records without mapping them to metrics). It is unique to mx-chain-vm-common-go's role as the metric translator for the entire DRWA compliance pipeline.

### Why This Gap Matters for Regulatory Compliance

MiCA Article 45 and equivalent regulations require that every transfer denial be attributed to a specific, documented compliance rule. A denial code that exists in the vocabulary but is excluded from `drwaDenialMetric` is invisible to the gate metrics — downstream monitoring and reporting tools have no automated signal to track it. This is not a theoretical risk — it is the exact failure mode that occurs every time a new denial reason is added to mx-chain-core-go without a corresponding entry in `drwaDenialMetric` and a corresponding test in mx-chain-vm-common-go.

The fix is a 10-line test that enforces the exhaustiveness contract between mx-chain-core-go and mx-chain-vm-common-go. Without this test, the regulatory gap grows silently with every new denial code added to the upstream vocabulary.